

Représentations 3D de Cartes

INF8422 : Perception Robotique et Intelligence Spatiale

Représentations 3D sparse et denses

Prof. Pierre-Yves Lajoie

**POLYTECHNIQUE
MONTREAL**

Agenda

1. Décrire la carte

Occupation, élévation, primitives, sémantique.

2. Stocker les données

Voxels, tables de hachage et octrees.

3. Représenter les surfaces

ESDF, TSDF et maillages.

4. Choisir selon le contexte

Environnement, dynamique et tâche.

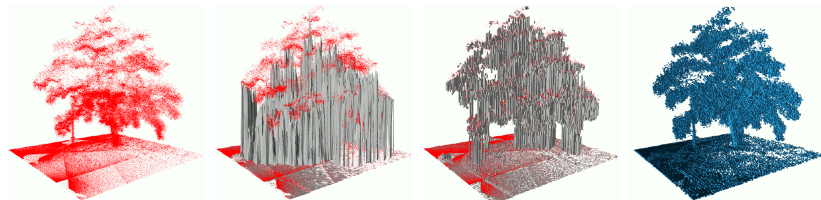
5. Passer à l'action

Recherche de chemin, marges, optimisation et exploration.

Représentations de Surfaces et Volumes

Qu'est-ce qu'une Carte ?

La **même scène** peut être représentée de multiples façons selon la tâche :



Un même arbre scanné : nuage de points · carte d'élévation · multi-level surface · voxels (Hornung et al., OctoMap 2013)

Choix de représentation selon la tâche :

Tâche	Représentation idéale
Localisation	Points sparse, landmarks
Navigation autonome	Grille d'occupation / ESDF
Inspection de surface	Maillage (mesh) dense
Planification de trajectoire	ESDF (gradients de distance)
Simulation / AR	Maillage texturé
Compréhension sémantique	Carte sémantique + objets

Pas de représentation universelle

Chaque représentation est un compromis entre mémoire, vitesse de construction, vitesse d'accès, et précision. Les systèmes modernes combinent souvent plusieurs représentations en parallèle.

Les 3 Questions Fondamentales

Q1 – Quoi estimer ?

Occupation ou **Distance** ?

- P(occupé) → navigation, exploration
- SDF/TSDF → reconstruction surface, fusion
- ESDF → planification (gradients de distance)

Q2 – Comment représenter ?

Explicite ou **Implicite** ?

- **Explicite** : géométrie directe (triangles, points)
- **Implicite** : fonction $f(\mathbf{x})$ dont le niveau zéro est la surface

Q3 – Quelle structure de données ?

Grille, Arbre ou **Table de hachage** ?

- Grille uniforme : simple, $O(1)$ accès, $O(N^3)$ mémoire
- Octree : hiérarchique, multi-résolution, $O(\log N)$
- Hash map spatiale : $O(1)$ amorti, domaine extensible. Collisions de hachage et surcoût de stockage.

Explicite vs Implicite – Comment Décrire la Géométrie ?

Représentation Explicite

La géométrie est **directement encodée** :

- Triangles, polygones et points
- On peut itérer sur la surface
- Accès direct aux éléments géométriques

Avantages : rendu rapide, modification directe

Inconvénients : connectivité à maintenir ; requêtes spatiales nécessitant une structure adaptée

Deux façons de décrire une même surface :

	Explicite	Implicite
Surface stockée	Points, triangles	Ensemble des zéros de f
Requête typique	Parcourir les éléments géométriques	Évaluer $f(\mathbf{x})$

Représentation Implicite

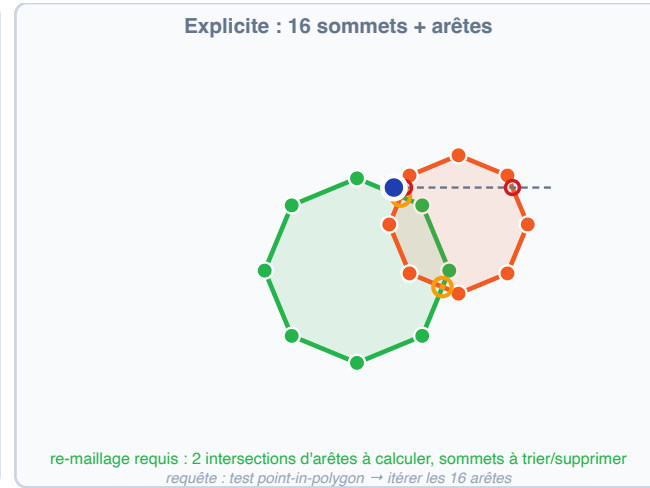
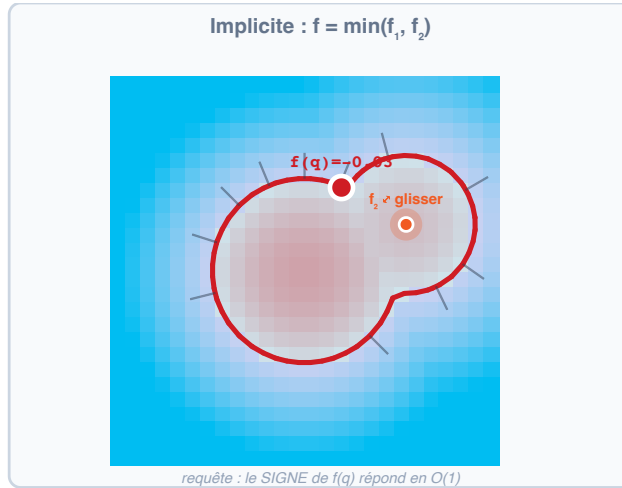
La surface est le **zéro-niveau d'une fonction** $f(\mathbf{x})$:

$$\mathcal{S} = \{\mathbf{x} \mid f(\mathbf{x}) = 0\}$$

Avantages : topologie flexible, interpolation naturelle, gradients disponibles
Inconvénients : extraction de surface nécessaire (e.g. Marching Cubes)

Explicite vs Implicite – Visualisation Interactive

Déplacez et déformez les objets sur la carte 2D, puis « extrayez » le maillage explicite.



1 forme **u union** $\cap$ inter. $\setminus$ soustr. $\cup$ lisse r 1.00 iso 0.0 sommets 8 [Extraire le maillage](#)

cliquer = point de requête · glisser f_2 (panneau gauche)

Grille d'Occupation 2D – la Carte Classique

La **grille d'occupation 2D** est la représentation historique et la plus répandue en robotique mobile (Elfes & Moravec, 1985).

- L'espace au sol est divisé en cellules carrées m_k
- Chaque cellule stocke $P(m_k = \text{occupé}) \in [0, 1]$
- Trois états utiles : **libre** / **occupé** / **inconnu**

Omniprésente en pratique :

- **Costmaps** de ROS (`nav2` , `move_base`)
- Planification 2D (A*, Dijkstra)
- Aspirateurs, AGV d'entrepôt, robots de service

Du 2D au 3D

Le même formalisme probabiliste (log-odds, modèle de capteur) se **généralise directement** en 3D :

$$\text{grille 2D } (i, j) \longrightarrow \text{grille 3D } (i, j, k)$$

Seule la dimension change ; les mises à jour restent identiques. Les prochaines slides traitent le cas **3D** (plus général), mais tout s'applique au cas 2D.

En 2D la mémoire est en $O(N^2)$; en 3D elle explose ($O(N^3)$) $\rightarrow$ d'où les structures creuses (octree, hashing) vues plus loin.

Cartes d'Occupation (*Occupancy Maps*)

Principe : diviser l'espace en cellules m_k et estimer :

$$P(m_k \mid z_{1:t}, x_{1:t})$$

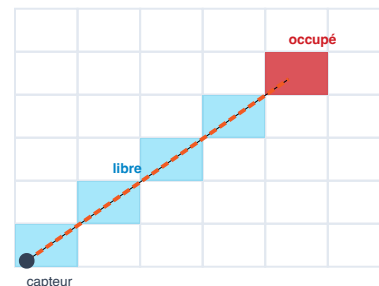
Hypothèse d'indépendance (entre cellules) :

$$P(m \mid z_{1:t}) = \prod_k P(m_k \mid z_{1:t})$$

Chaque cellule est classifiée : **Libre** / **Occupé** / **Inconnu**

Modèle de capteur :

- Le rayon traverse → cellule libre
- Le rayon s'arrête → cellule occupée



Modèle inverse : cellules traversées = libres, impact = occupé

Avantages :

- Simple et efficace
- Distingue explicitement libre / occupé / inconnu
- Essentiel pour l'exploration (trouver les zones inconnues)
- Correction si l'espace devient libre (objet bougé)

Inconvénients :

- Discrétisation fixe (pas de multi-résolution)
- Ne capture pas la corrélation spatiale
- $O(N^3)$ mémoire pour une grille 3D uniforme

Mise à jour Log-Odds

Log-odds ratio :

$$l(x) = \log \frac{p(x)}{1 - p(x)}$$

Propriété clé : la mise à jour devient **additive**, ce qui est beaucoup plus stable.

Mise à jour log-odds :

$$l(m_k \mid z_{1:t}) = l(m_k \mid z_{1:t-1}) + l(m_k \mid z_t) - l_0$$

où $l_0 = \log \frac{P(m_k)}{1-P(m_k)}$ est le log-odds prior.

Modèle de capteur :

$$l_{free} < 0 \quad (\text{rayon passe})$$

$$l_{occ} > 0 \quad (\text{rayon s'arrête})$$

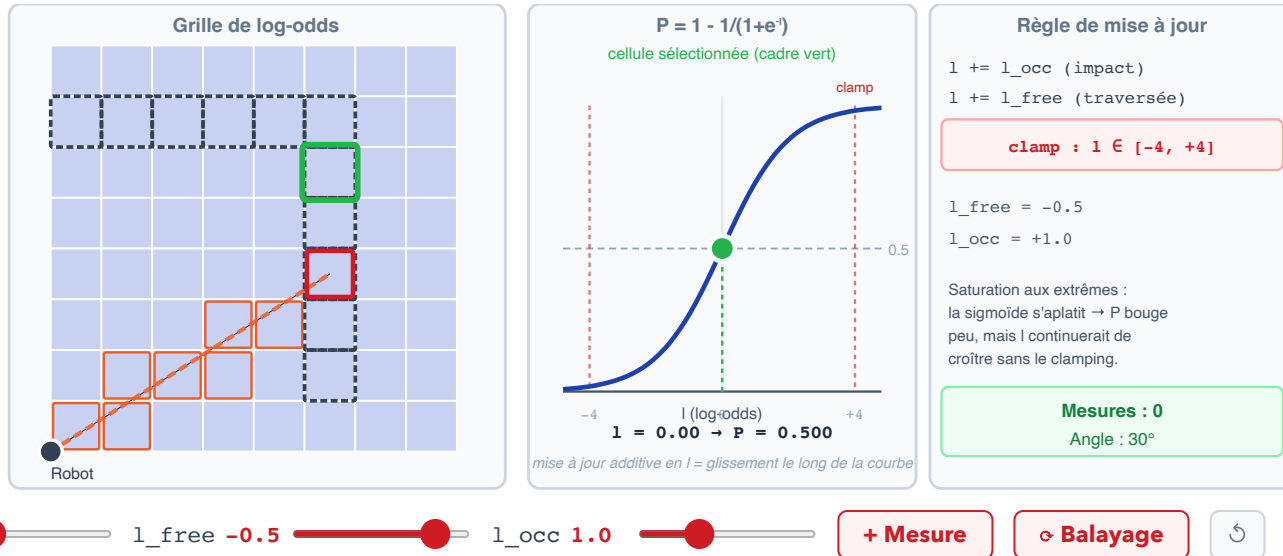
Conversion inverse pour retrouver la probabilité d'occupation :

$$p(m_k) = 1 - \frac{1}{1 + \exp(l(m_k))}$$

Clamping : $l \in [l_{min}, l_{max}]$ pour éviter la saturation.

Mise à jour Log-Odds – Visualisation Interactive

Chaque rayon ajoute l_{free} aux cellules traversées et l_{occ} à l'impact. Suivez une cellule le long de la sigmoïde $P = 1 - 1/(1 + e^{-l})$ et l'effet du *clamping*.



Cartes d'Élévation (2.5D)

Principe : une grille horizontale (x, y) où chaque cellule stocke une **seule hauteur** $z = h(x, y)$.

$$h : \mathbb{R}^2 \rightarrow \mathbb{R}$$

« **2.5D** » : entre la 2D (grille plane) et la 3D (volume complet). On capture le relief du sol sans le coût d'une grille de voxels 3D.

Avantages :

- Très **compact** et rapide (une valeur par cellule)
- Idéal pour la **navigation au sol** (rovers, robots à pattes)
- Gradients de pente directement disponibles

Limite fondamentale – les surplombs : Une seule hauteur par cellule → **impossible** de représenter un pont, un tunnel, une branche (deux surfaces à la même (x, y)).

Multi-Level Surface (MLS) maps

Extension : stocker **plusieurs** hauteurs (surfaces) par cellule → gère les surplombs, tout en restant bien plus léger qu'une grille 3D dense.

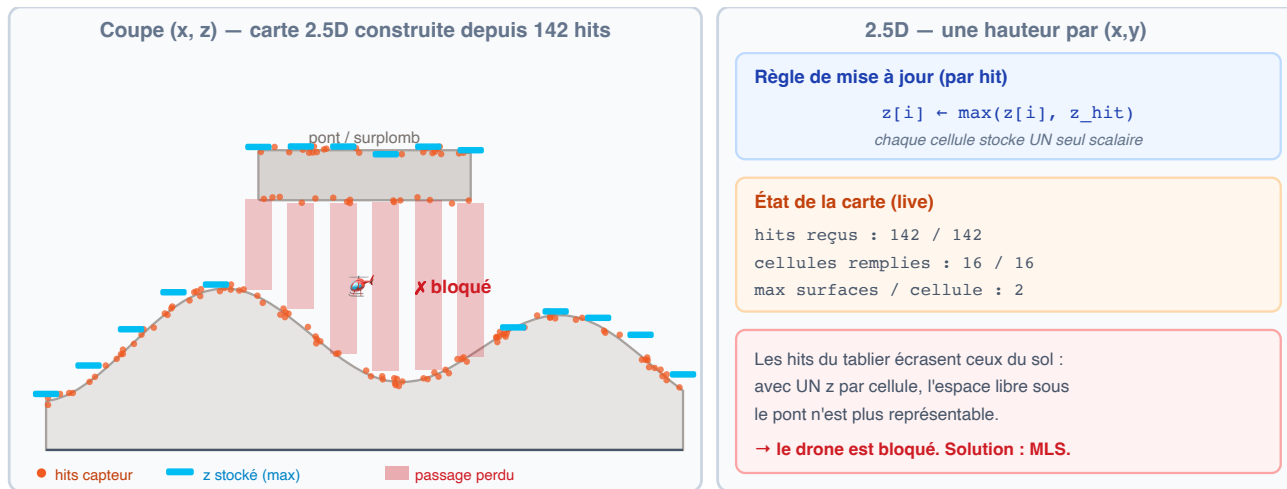
Systèmes

GridMap (ANYbotics, robots quadrupèdes), **Elevation Mapping** (locomotion sur terrain accidenté), cartes DEM (télédétection, photogrammétrie).

Représentation **classique** en robotique terrestre, souvent préférée aux voxels 3D quand le monde est essentiellement un terrain.

Cartes d'Élévation 2.5D – Visualisation Interactive

► **Scanner** : la carte se construit hit par hit. Mêmes mesures, deux règles de stockage, en 2.5D le tablier du pont écrase le sol ; en MLS les hits se regroupent en surfaces.



► Scanner

2.5D

MLS

✓ surplomb

terrain

mêmes mesures, deux règles de stockage – basculez 2.5D ↔ MLS pendant ou après le scan

Cartes de Plans et de Primitives

Idée : dans un environnement **structuré** (intérieurs, usines), la géométrie se résume à quelques **primitives** : plans (murs, sol, plafond), cylindres, sphères.

Ajustement de plan (par cellule ou région) :

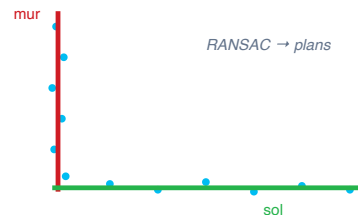
$$\mathbf{n}^\top \mathbf{p} + d = 0, \quad \|\mathbf{n}\| = 1$$

, où d est la distance du plan de l'origine.

Extraction : **RANSAC** (robuste aux mesures aberrantes) ou croissance de région sur les normales.

Avantages :

- Extrêmement **compact** (un mur = 4 nombres $(\mathbf{n}, d)$ vs millions de points)
- **Robuste** : moyenne beaucoup de mesures bruitées
- Contraintes fortes pour le SLAM (plans comme landmarks)



SLAM planaire

Utiliser les plans comme **contraintes/landmarks** dans le graphe de facteurs : très efficace en intérieur. Ex : **CPA-SLAM**, LiDAR planaire (LOAM = arêtes + plans).

Limité aux environnements **structurés**, inadapté à la végétation ou aux terrains semi- ou non-structurés.

Cartes Sémantiques

Idée : enrichir chaque élément géométrique (voxel, point, triangle) d'un **label** : la carte dit où **et** quoi.

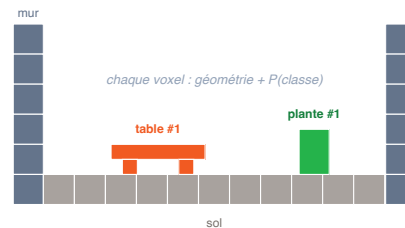
Fusion bayésienne des labels (par cellule) : un réseau de segmentation prédit $p_k(c \mid \text{image})$ par pixel ; reprojeté sur le voxel, on accumule :

$$P_k(c) \propto P_{k-1}(c) \cdot p_k(c \mid \text{image})$$

La même logique multiplicative que le log-odds, mais **multi-classe**.

Trois niveaux :

- **Sémantique** : classe par cellule (mur, chaise...)
- **Instance** : chaise #1 $\neq$ chaise #2
- **Panoptique** : différencie entre *stuff* (mur, sol) et *things* (objets)



Systèmes

Kimera-Semantics (maillage sémantique), **Panoptic Multi-TSDFs** (une sous-carte TSDF par objet). Les représentations TSDF et maillage sont détaillées plus loin.

Ce que ça débloque

Navigation contextuelle (« va à la cuisine »), manipulation (saisir *la tasse*), inspection ciblée, cartes long-terme (les objets bougent, les murs non).

La **géométrie** vient toujours des représentations de ce cours, le réseau de neurones ne fournit que les labels.

Voxels

Voxel = "Volumetric Pixel": discrétisation régulière de l'espace 3D.

Chaque voxel (i, j, k) contient une propriété :

- $P(\text{occupé}) \rightarrow$ carte d'occupation
- Valeur TSDF $\rightarrow$ reconstruction de surface
- Couleur, label sémantique, etc.

Accès direct : indexation $(i, j, k) \rightarrow O(1)$

Voisinage immédiat : les 6 (face), 18 (arête) ou 26 (coin) voisins

Problème : $O(N^3)$ mémoire – la plupart des voxels sont vides !

Exemple : $100\text{m} \times 100\text{m} \times 10\text{m}$ à 5 cm $\rightarrow \sim 8 \times 10^8$ voxels

Observation clé : les surfaces 3D forment une **variété 2D** dans l'espace 3D $\rightarrow$ la quasi-totalité des voxels sont vides.

Stocker seulement les voxels utiles

Plutôt qu'une grille dense $O(N^3)$, on n'alloue que les S voxels nécessaires ($S \ll N^3$) via des **structures creuses** : octree ou table de hachage spatiale.

Les prochaines diapos comparent les **tables de hachage spatiales** et les **octrees**.

Le Défi des Données Denses

Exemple concret :

Carte 100m × 100m × 10m à résolution 5 cm :

$$M = \frac{100 \times 100 \times 10}{0.05^3} \approx 8 \times 10^8 \text{ voxels}$$

À 4 bytes par voxel → **3.2 Go** de RAM juste pour la grille, **vide pour la plupart**.

Observation clé :

L'environnement réel est **sparse** :

- La majorité de l'espace est vide (air)
- Les surfaces 3D sont des **variétés 2D** dans un espace 3D
- La reconstruction privilégie les surfaces ; la navigation a aussi besoin de l'espace libre

Objectif : stocker **uniquement** ce qui est nécessaire.

Compromis fondamental

Grille uniforme : accès $O(1)$, mémoire $O(N^3)$

Octree : accès $O(\log N)$, mémoire $O(S)$

Hash map spatiale : accès $O(1)$ amorti, mémoire $O(S)$

N : résolution par axe ; S : éléments stockés, souvent $S \ll N^3$.

Pas de solution universelle. Dépend aussi du matériel (CPU/GPU) et de la tâche.

Hash Maps Spatiales

Principe : diviser l'espace en **blocs** (groupes de voxels) et les mapper via une fonction de hachage.

Fonction de hachage spatiale :

$$h(i, j, k) = (i \cdot p_1 \oplus j \cdot p_2 \oplus k \cdot p_3) \mod M$$

où p_1, p_2, p_3 sont des grands nombres premiers.

Structure :

- Clé : coordonnées du bloc (i, j, k)
- Valeur : pointeur vers le bloc de voxels

Voxel Hashing (Niessner 2013) :

- Blocs de $8^3 = 512$ voxels (GPU-friendly)
- Dense à l'intérieur de chaque bloc (tableau linéaire)
- Sparse au niveau des blocs (hash map)

Avantages :

- Accès $O(1)$ amorti (comme une table de hachage classique)
- Domaine **infini** (pas de bornes pré-définies)
- Mémoire : alloue seulement les blocs observés

Considérations :

- Pas de localité en mémoire: pas d'accès par segment
- Collisions : rares avec bonne fonction de hachage
- Granularité du bloc : compromis accès vs overhead mémoire

Exemple : Voxblox

Voxblox utilise des blocs de voxels indexés par hachage pour la cartographie en temps réel.

Arbres et Octrees

Principe : décomposition **hiérarchique** récursive de l'espace.

Octree : chaque octant est divisé en 8 enfants de taille $1/2$.

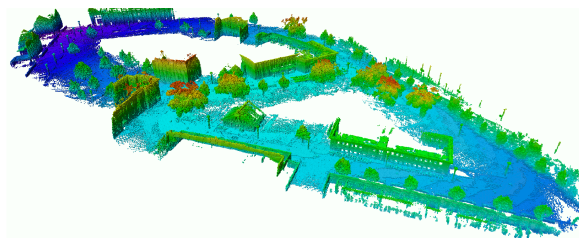
Un nœud est **subdivisé** uniquement si son contenu est hétérogène (surface + espace libre).

Avantages :

- **Multi-résolution intrinsèque** : zones uniformes restent grossières
- Compression efficace des zones libres/occupées uniformes
- Recherche de voisinage rapide

Inconvénients :

- Accès $O(\log N)$ (descente de l'arbre)
- Moins GPU-friendly que le hash map



Octree du campus de Fribourg ($292 \times 167 \times 28$ m, 20 cm), hauteur en couleur (OctoMap 2013)

OctoMap (Hornung 2013) :

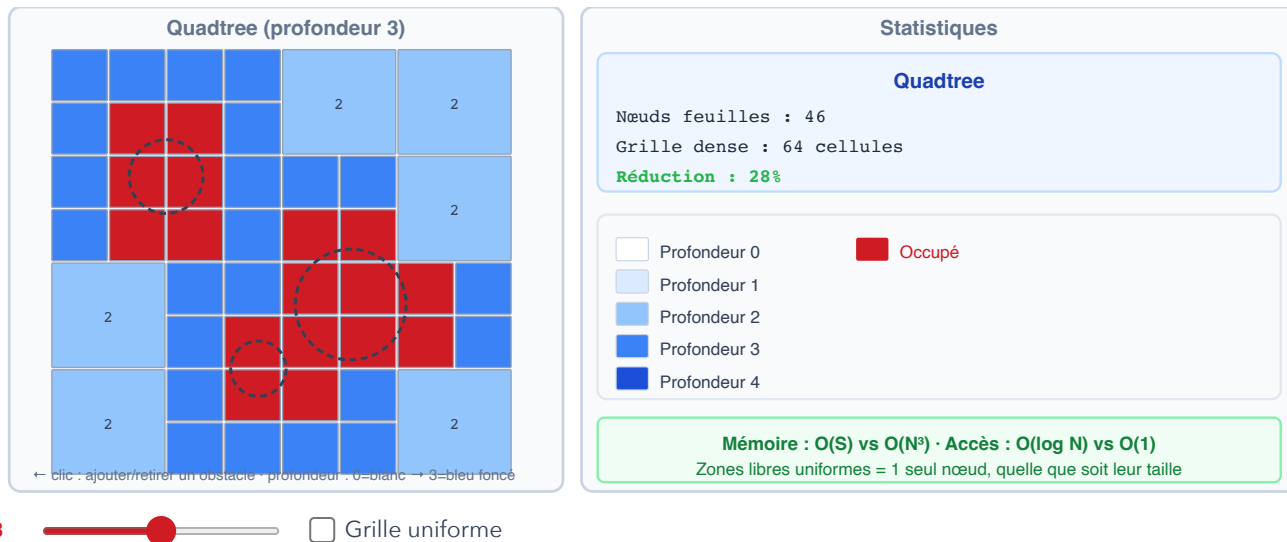
- Octree probabiliste avec log-odds
- **Pruning** : fusionner les enfants si même valeur → compression
- Open source, intégré à ROS
- Représentation standard pour la navigation mobile

Résolution adaptative

OctoMap permet de requêter à différentes résolutions : $2^k \times$ résolution minimale, en remontant l'arbre.

Arbres et Octrees – Visualisation Interactive

Cliquez pour ajouter/retirer des obstacles : la subdivision ne descend que là où le contenu est **hétérogène*. Comparez le nombre de nœuds à celui d'une grille dense.



Surfaces Implicites

Rappel : surface = niveau zéro de $f : \mathbb{R}^3 \rightarrow \mathbb{R}$, avec $f < 0$ à l'intérieur, $f > 0$ à l'extérieur.

Toutes les fonctions implicites ne se valent pas :

- $f_1(x, y) = x^2 + y^2 - r^2$: le **signe** est correct, mais la valeur n'est pas dans les bonnes unités (des m²...)
- $f_2(x, y) = \sqrt{x^2 + y^2} - r$: la valeur **est** la distance signée au cercle

SDF (Signed Distance Function) :

$$f(\mathbf{p}) = \pm d(\mathbf{p}, \mathcal{S})$$

Normale gratuite :

$$\mathbf{n} = \frac{\nabla f}{\|\nabla f\|} \quad (\text{SDF régulière : } \|\nabla f\| = 1)$$

Pourquoi vouloir une SDF :

1. **Marge de sécurité directe** : $f(\mathbf{p}) > r_{robot} \iff$ pas de collision (r_{robot} est le rayon englobant du robot)
2. **Gradient = direction d'évitement** : ∇f pointe vers l'extérieur
3. **Topologie flexible** : l'union se décrit par $\min(f_1, f_2)$; le champ obtenu n'est pas forcément une distance exacte partout (comme dans l'animation explicite / implicite)

La suite logique

Deux champs de distance pratiques en robotique : l'**ESDF** (euclidienne, pour la planification) et la **TSDF** (projective et tronquée, pour la fusion).

ESDF – Distance à la Surface la Plus Proche

ESDF : champ de distance euclidienne signé.

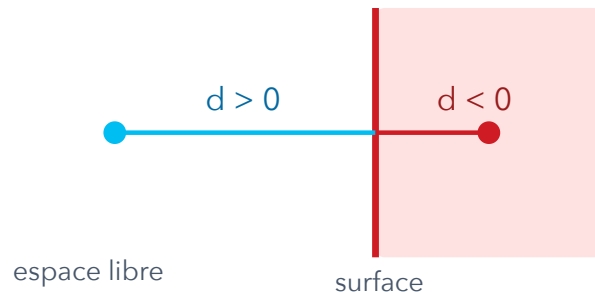
$$d(\mathbf{x}) = s(\mathbf{x}) \min_{\mathbf{p} \in \mathcal{S}} \|\mathbf{x} - \mathbf{p}\|_2$$

- Extérieur : $d > 0$
- Surface : $d = 0$
- Intérieur : $d < 0$

$\mathcal{S}$ est la surface des obstacles ; $s \in \{-1, +1\}$ indique le côté.

Occupation → distance

L'occupation répond « **est-ce occupé ?** » ; l'ESDF répond « **à quelle distance est l'obstacle ?** »



Pour planifier :

- Comparer d au **rayon du robot + marge**.
- Utiliser ∇d pour s'éloigner des obstacles.
- Interpoler la grille pour interroger une position continue.

Sur une grille, la précision dépend de la résolution et de la convention de surface.

Construire l'ESDF – Initialiser, Puis Balayer X

1. Définir la grille : domaine et taille de voxel h .

2. Initialiser les sites :

- obstacle observé $\rightarrow 0$;
- autres sites $\rightarrow \infty$;
- conserver séparément le masque **observé / inconnu**.

3. Pour chaque ligne suivant X :

Passer **avant** $\rightarrow$

$$g_i \leftarrow \min(g_i, g_{i-1} + 1)$$

Passer **arrière** $\leftarrow$

$$g_i \leftarrow \min(g_i, g_{i+1} + 1)$$

Une ligne – distances en nombre de voxels

Initial	∞	∞	0	∞	∞	∞	0
Avant $\rightarrow$	∞	∞	0	1	2	3	0
Arrière $\leftarrow$	2	1	0	1	2	1	0

Pourquoi deux sens ?

L'obstacle le plus proche peut se trouver **à gauche ou à droite**. Après les deux passes, chaque site connaît sa distance à un obstacle **sur cette ligne**.

Pour combiner les axes, on garde ensuite les **carrés** : $D_X = g^2$.

Sans obstacle sur une ligne, ses valeurs restent à ∞ à cette étape.

Construire l'ESDF – Compléter Y, Z et le Signe

4. Combiner les axes en distances au carré

Pour chaque colonne Y :

$$D_{XY}(x, y) = \min_j [D_X(x, j) + (y - j)^2]$$

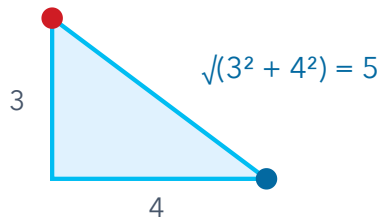
- Passe **avant** : sélectionner les candidats utiles.
- Passe **arrière** : évaluer le minimum à chaque site.
- En 3D, répéter la même opération suivant Z.

5. Prendre la racine et convertir en mètres

$$d_{\text{ext}} = h\sqrt{D_{XYZ}}$$

Trois axes, un travail linéaire par ligne : $O(M)$ pour M voxels au total.

La combinaison est euclidienne



6. Distinguer intérieur et extérieur

- Libre observé : distance **positive** aux sites occupés.
- Occupé : distance **négative** aux sites libres, obtenue par une seconde transformée.
- Inconnu : **pas de distance exploitable** pour naviguer.

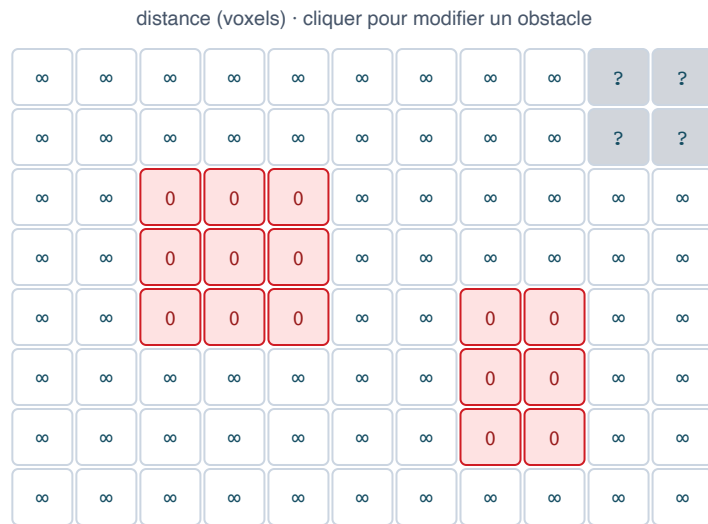
Convention discrète : distances aux **centres de la classe opposée**.
Le zéro est entre les classes.

Lorsque la carte change

Recalculer les distances affectées. La démonstration suivante rejoue les passes sur toute la petite grille.

ESDF – Les Passes Avant et Arrière

Suivez $X \rightarrow Y \rightarrow$ **racine et signe**. Cliquez sur une cellule pour ajouter ou retirer un obstacle, puis observez le nouveau champ.



Rouge : obstacle · bleu : valeur finie · gris : inconnu

► Lecture

⏮ Ligne suivante

↺ vitesse

Après ajout ou retrait d'un obstacle, la démonstration rejoue les passes sur la grille.

0 / ∞ → $X \rightleftharpoons$ → $Y \updownarrow$ → $\sqrt{\quad}$ + signe

Initialiser

Obstacles : 0. Autres sites : ∞.

Le gris reste inconnu ; il ne devient pas libre.

En 3D : ajouter les passes sur Z avant la racine.

Convention discrète : distance aux centres de la classe opposée. Le zéro se situe entre les classes.

TSDF – Champ de Distance Tronqué

Chaque voxel stocke une **distance projective signée** F , tronquée et fusionnée, ainsi qu'un **poids** W .

- signe **positif** → devant la surface (espace libre)
- signe **négatif** → juste derrière la surface observée (côté intérieur)

Exemple – un mur vu par une caméra de profondeur :

Position du voxel	Valeur
Sur le mur	0
5 cm devant (air)	+0.05
5 cm dans le mur (solide)	-0.05
2 m dans la pièce ($\delta = 10$ cm)	+0.10 (plafonné)

Différence avec ESDF : distance mesurée **le long du rayon** du capteur (projective), pas la distance Euclidienne globale.

Troncature de la distance projective :

$$f(\mathbf{p}) = \max(-\delta, \min(\delta, d_{\text{measure}} - d_{\text{point}}))$$

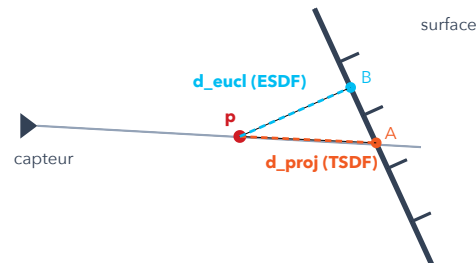
La distance est plafonnée à $\pm\delta$. Les voxels à plus de δ derrière la surface ne sont pas intégrés.

Mise à jour pondérée (KinectFusion) :

$$F_{k+1}(\mathbf{x}) = \frac{W_k(\mathbf{x}) F_k(\mathbf{x}) + w_k d_k}{W_k(\mathbf{x}) + w_k}$$

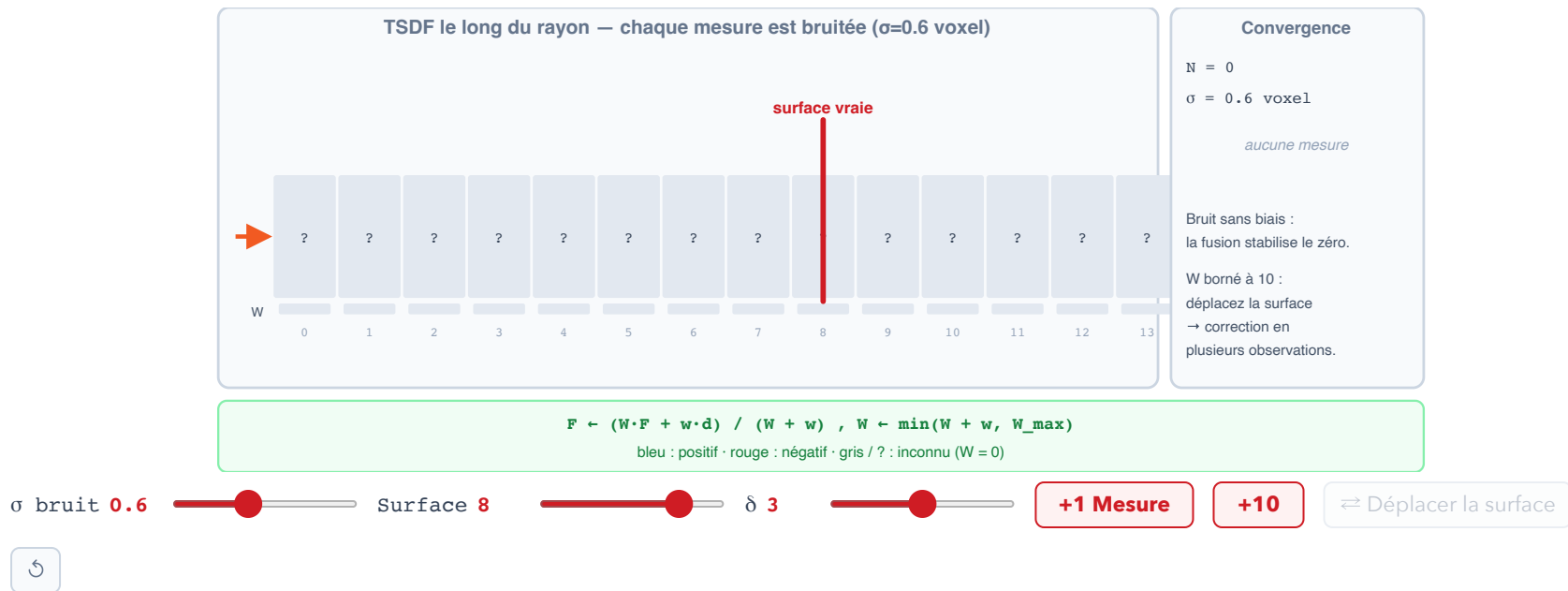
f, d : valeur d'une mesure isolée · F, W : valeur/poids fusionnés en mémoire · w : poids d'une mesure

Inconnu : $W = 0$. Aucune surface ne doit être déduite d'un voxel sans observation.



TSDF – Visualisation Interactive

Les mesures sont **bruitées** : la moyenne pondérée $F \leftarrow (WF + wd)/(W + w)$ lisse le passage par zéro au fil des observations ; W_{max} garde le voxel corrigé. (F, W : valeur et poids déjà fusionnés au voxel · d, w : valeur et poids de la nouvelle mesure)



TSDF pour la Fusion, ESDF pour la Planification

Le **TSDF fusionne les observations** ; l'**ESDF fournit les distances** utiles au planificateur.

1. Réduire le bruit

Le TSDF conserve une **moyenne pondérée** des distances et un poids cumulé. Les observations répétées stabilisent la surface.

Les biais, erreurs de pose et valeurs aberrantes demandent un traitement supplémentaire.

2. Intégrer rapidement

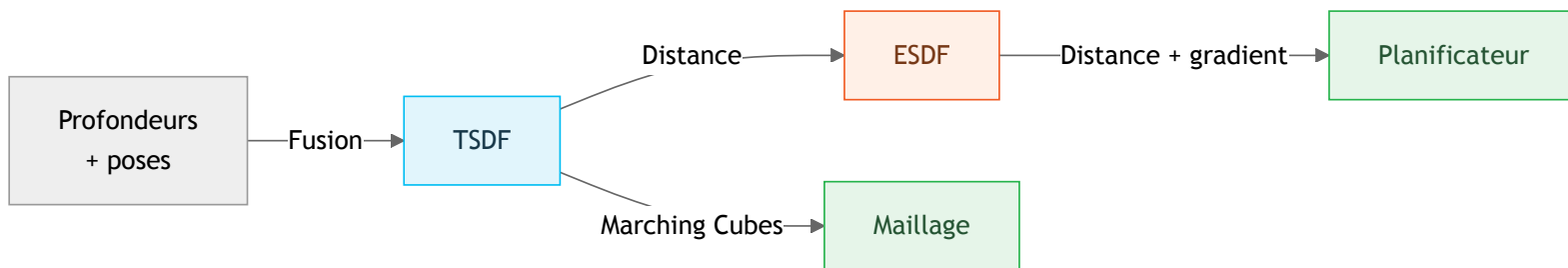
L'écart de profondeur se calcule **le long du rayon**, sans rechercher l'obstacle le plus proche dans toutes les directions.

Mise à jour locale et parallélisable ; l'ESDF ajoute un calcul de distance sur la carte.

3. Reconstruire la surface

L'interpolation de $F = 0$ situe la surface entre les centres des voxels. Marching Cubes en extrait un maillage.

La précision dépend des mesures et des poses. Une ESDF peut aussi avoir un zéro sous-voxel.



L'ESDF se met à jour de façon **incrémentale** : seules les distances affectées sont recalculées. Un faux obstacle non filtré peut influencer toute une région.

TSDF → ESDF : Partir de la Surface, Étendre les Distances

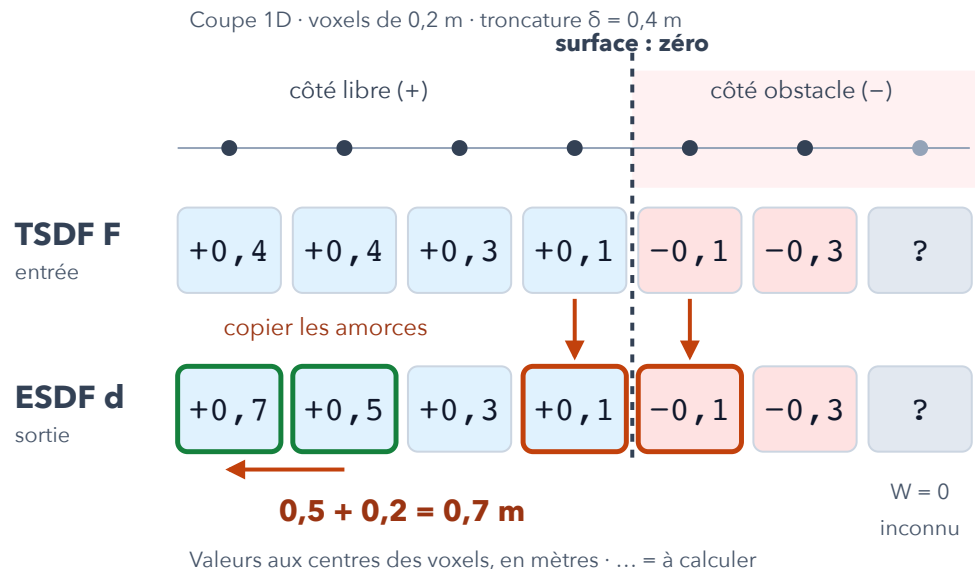
Le TSDF localise la **surface** ; on en déduit un champ de **distances pour planifier**.

1 Lire le TSDF

2 Fixer les amorces

3 Propager

4 Lire l'ESDF



Une distance utile loin de la surface

À gauche, l'ESDF donne +0,7 m jusqu'au mur, là où le TSDF plafonnait à +0,4 m.

Même surface, même signe : le champ de distance s'étend dans le domaine observé.

? reste inconnu

On n'invente pas de distance dans les voxels sans observation.

En 2D / 3D : propager la distance à la surface la plus proche dans toutes les directions.

Exemple idéal : mur plan, rayon perpendiculaire. En général, les amorces TSDF sont approximatives.

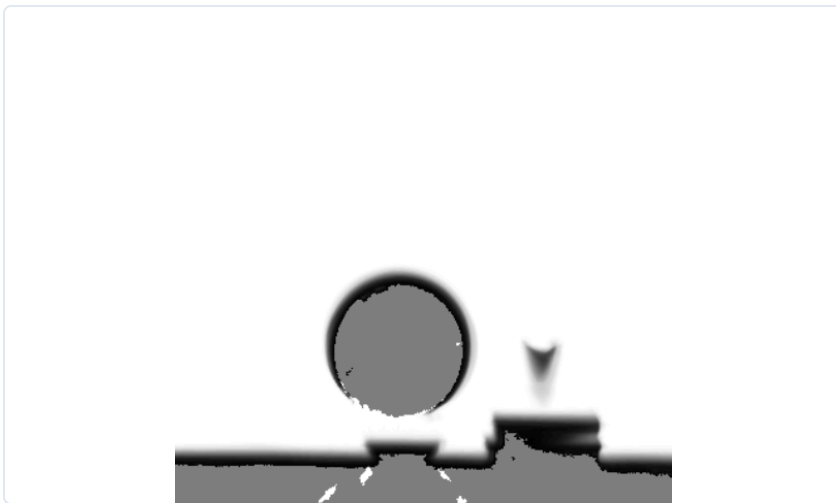


vue 5 / 5



TSDF et ESDF – Visualisations dans la Littérature

TSDF – KinectFusion



Variation dans une bande près de la surface.

Blanc : distance positive plafonnée.

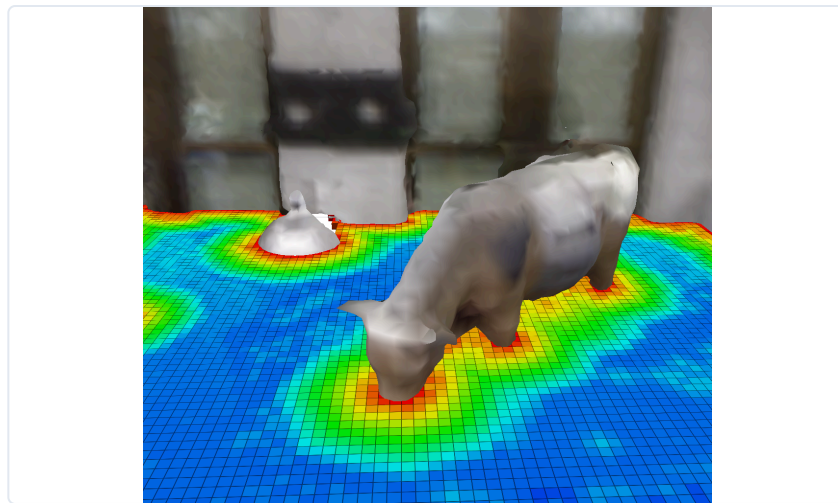
Gris : voxels sans mesure valide.

Newcombe et al., **KinectFusion**, ISMAR 2011.

Figure 4, panneau gauche.

Scènes différentes ; couleurs et échelles propres à chaque publication.

ESDF – Voxblox



Distance aux surfaces dans l'espace libre.

Coupe colorée : valeurs de l'ESDF.

Maillage 3D : repère de la scène reconstruite.

Oleynikova et al., **Voxblox**, prépublication 2016.

Figure 1, arXiv:1611.03631v1.

Comparaison : Occupation vs Implicite

Critère	Carte d'Occupation	Surface Implicite (TSDF/ESDF)
Nature	Probabiliste ($P \in [0, 1]$)	Géométrique (distance signée)
Construction	Fusion probabiliste des observations	TSDF : fusion ; ESDF : transformée de distance
Requêtes	Probabilité ou classe après seuillage	Valeur interpolée et gradient là où il existe
Libre/Inconnu	État ou historique d'observation	Poids TSDF / masque d'observation à conserver
Surface	Frontière entre classes discrètes	Niveau zéro interpolé
Usage typique	Navigation, exploration	Reconstruction, physique, planification

Quand choisir l'occupation ?

Robot autonome devant distinguer espace libre / inconnu / occupé. Exploration : le robot doit savoir où il n'a pas encore regardé.

Quand choisir TSDF/ESDF ?

Reconstruction 3D de précision (inspection, AR), ou planification de trajectoire nécessitant des distances et des gradients interpolés.

Maillages (*Meshes*)

Une surface représentée par des **sommets reliés en triangles**.

$$\mathbf{V} \in \mathbb{R}^{n_v \times 3}, \quad \mathbf{F} \in \mathbb{N}^{n_f \times 3}$$

- **Sommets** : coordonnées 3D.
- **Faces** : trois indices de sommets par triangle.
- **Connectivité explicite** entre les éléments.
- Normales, couleurs et textures pour le rendu.

Usages : visualisation, inspection, simulation et export vers les outils 3D.

Depuis un volume fusionné

TSDF → Marching Cubes → maillage.

On peut extraire le maillage sur demande ou mettre à jour les blocs modifiés.



1 000 triangles : la surface est un assemblage de faces planes.

Garland et Heckbert, SIGGRAPH 1997, fig. 9.
Surface Simplification Using Quadric Error Metrics

Du TSDF au Maillage – Marching Cubes

Extraire l'isosurface $F = 0$, cube par cube.

1. Parcourir les cellules

Chaque cube relie **8 échantillons voisins** du TSDF.

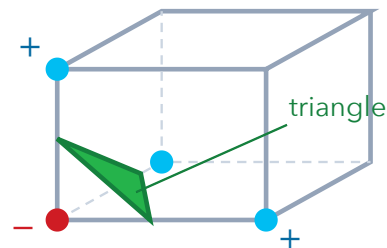
2. Trouver les passages par zéro

Sur une arête de signes opposés, interpoler le sommet :

$$\mathbf{v} = \mathbf{p}_A + \frac{F_A}{F_A - F_B}(\mathbf{p}_B - \mathbf{p}_A)$$

3. Relier les sommets en triangles

Les 8 signes forment un code binaire : **256 configurations** → une table de triangles.



4. Calculer les normales

La direction du gradient de F donne la normale, utile pour l'éclairage.

5. Assembler le maillage

Réunir sommets et faces ; partager les sommets communs, retirer les triangles dégénérés, exporter (**PLY / OBJ**).

Lorensen et Cline, SIGGRAPH 1987 – Marching Cubes: A High Resolution 3D Surface Construction Algorithm.

Marching Cubes – Du Champ aux Facettes

À gauche, l'analogie **2D (Marching Squares)** : signes → arêtes traversées → segments interpolés. À droite, **Marching Cubes en 3D** produit des triangles.



Ces champs synthétiques illustrent l'extraction d'une isosurface. Dans un TSDF fusionné, on extrait le niveau $F = 0$.

Conversions – Une Carte, Plusieurs Usages

Depuis les observations et leurs poses :

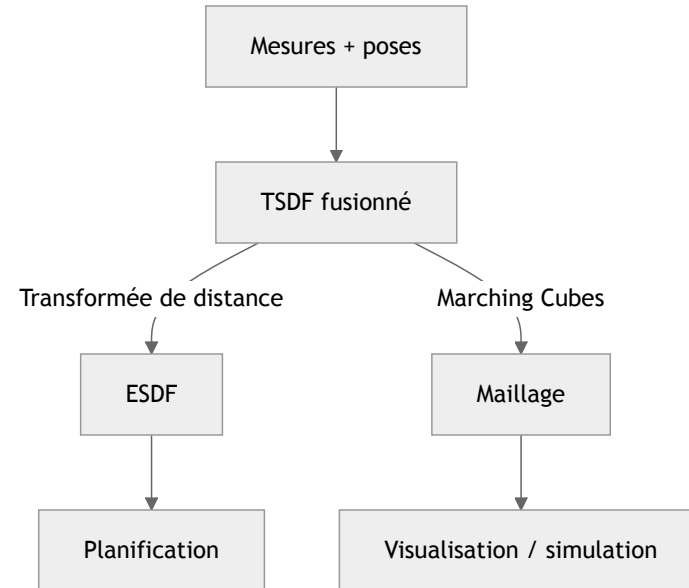
- Fusion probabiliste → **occupation**.
- Fusion des écarts de profondeur → **TSDF**.

Depuis la carte fusionnée :

- Occupation ou TSDF → **ESDF** : distances pour planifier.
- TSDF → **Marching Cubes** → maillage : surface pour visualiser ou simuler.

Des sorties complémentaires

Le maillage décrit la **surface** ; l'ESDF décrit les **distances dans le volume**.



Considérations Pratiques

Environnement : Structuré vs Non-Structuré

Structuré (ex: usines, entrepôts)

- Géométrie **connue ou simple** : plans, cylindres
- Cartes CAD souvent suffisantes
- **Représentations légères** (plans paramétriques, landmarks fixes)
- Localisation par correspondance CAD-LiDAR

Non-Structuré / Inconnu (ex: forêt, décombres)

- Forme arbitraire → représentations denses nécessaires
- Information **libre/inconnu** cruciale pour l'exploration sûre
- **Voxels/Octrees** : distinguent explicitement les zones non observées
- Points / maillages seuls : l'absence de surface ne prouve pas que l'espace est libre

Exploration autonome

Pour un robot explorateur (ex: rover Mars, drone de secours), la distinction **libre / inconnu** est critique : le robot ne doit pas naviguer dans l'inconnu comme si c'était sûr.

Les cartes d'occupation (OctoMap) excellent ici. Dans un TSDF, les poids indiquent les observations disponibles ; le planificateur doit aussi gérer l'inconnu.

Environnement Dynamique – Les Fantômes de la Carte

Un objet mobile quitte sa position, mais ses anciennes observations persistent : la carte contient un **faux obstacle**, ou *ghost*.



Intégration naïve

Les traces s'effacent lentement quand l'espace est revu libre.

Masquage

Exclure les mesures de l'objet mobile avant la fusion.

Correction

Retirer les traces contredites par de nouvelles observations.

Quelle Représentation pour Quelle Tâche ?

Tâche	Repr. recommandée	Pourquoi
Localisation	Points et primitives	Léger, scan-matching rapide
Navigation / Planification	Grille d'occupation + ESDF	Libre/inconnu, gradients de distance
Reconstruction 3D	TSDF + Marching Cubes → Mesh	Lissage, topologie de surface
Visualisation	Maillage texturé	Rendu GPU direct
Simulation physique	Maillage watertight, SDF	Détection de collisions, FEM
Exploration autonome	OctoMap (octree probabiliste)	Multi-résolution, inconnu explicite
Manipulation	Maillage + SDF	Surfaces précises, gradients
Compréhension sémantique	Voxels sémantiques, Panoptic	Fusion sémantique et géométrique

Systèmes hybrides modernes

Les systèmes robustes combinent plusieurs représentations : **TSDF** pour la fusion → **ESDF** pour la planification ; en parallèle, **TSDF** → **Mesh** pour la visualisation. nvblox (NVIDIA Isaac) fait les trois sur GPU.

Résumé – Représentations de Cartes

Géométrie

Représentation	Clé	Usage
Élévation / MLS	Hauteurs par cellule	Traversabilité
Primitives	Plans, cylindres	Environnements structurés
Maillage	Topologie explicite	Visualisation, physique

Volumes

Représentation	Clé	Usage
Grille d'occupation	P(occ), free/unknown	Navigation
TSDF	Distance projective	Fusion, reconstruction
ESDF	Distance euclidienne signée	Planification
Sémantique	Labels sur la géométrie	Compréhension de scène

Structures de données

Structure	Accès	Mémoire
Grille uniforme	$O(1)$	$O(N^3)$
OctoMap	$O(\log N)$	$O(S)$
Hachage spatial	$O(1)$ en moyenne	$O(S)$

Pas de solution universelle, le bon choix dépend du trio (**robot, environnement, tâche**).

Des Représentations à la Planification

La Carte au Service de l'Action

La carte n'est **pas une fin en soi**, elle sert la boucle :

percevoir → cartographier → planifier → agir

Le choix de représentation détermine **quels planificateurs sont possibles** :

Représentation	Planificateur	Exemple
Grille d'occupation	Recherche de graphe	A*, D* Lite (nav2)
ESDF	Optimisation de trajectoire	CHOMP, TrajOpt, MPPI
Carte d'élévation	Locomotion / footsteps	ANYmal, rovers
Mesh + SDF	Manipulation	Graspln, contact
Occupation (inconnu)	Exploration	Frontières

Les 3 questions, revisitées

Q1 (quoi estimer) prend ici tout son sens : la recherche de graphe se contente d'**occupé/libre**, l'optimisation continue exige la **distance et son gradient**.

Le critère qui compte

Une carte magnifique mais trop lente à requêter est inutile au planificateur : c'est le **temps de requête** ($d(\mathbf{x})$, ∇d , collision) qui dicte le choix, pas la beauté de la reconstruction.

Une carte, deux sorties

Les profondeurs fusionnées dans le TSDF peuvent alimenter un ESDF pour la trajectoire et un maillage pour l'inspection.

Recherche de Chemin sur Grille d'Occupation

Formulation : les cellules **libres** forment un graphe 8-connexte, coûts d'arête $\{1, \sqrt{2}\}$.

Dijkstra : explore par coût croissant $g(n)$ – des "cercles" concentriques depuis le départ. Optimal, mais visite beaucoup de nœuds.

A* : ajoute une heuristique vers le but :

$$f(n) = \underbrace{g(n)}_{\text{coût depuis départ}} + \underbrace{h(n)}_{\text{estimation vers but}}$$

Admissibilité : si h ne **surestime jamais** le vrai coût restant (ex : distance euclidienne), A* trouve un chemin de **même coût optimal** que Dijkstra, souvent en explorant moins.

Pourquoi 8-connexte ?

Les diagonales à $\sqrt{2}$ autorisent des déplacements à 45°. Le chemin reste contraint par les directions de la grille ; il peut être plus long qu'un chemin continu.

La grille donne un chemin **géométrique** en zigzag – sans dynamique ni lissage. L'optimisation de trajectoire, présentée après les méthodes de recherche, permet de le lisser.

A* vs Dijkstra – Visualisation Interactive

Lancez les **deux** algorithmes : même coût optimal, mais A* explore généralement moins de nœuds. Le réglage r_{robot} gonfle les obstacles via le champ de distance.

Grille d'occupation

← clic : ajouter / retirer un obstacle

Dijkstra vs A*

- Obstacle
- Zone gonflée (dist < r_{robot})
- Nœuds fermés (explorés)
- Chemin optimal

$f(n) = g(n) + h(n)$

Dijkstra : $h = 0$ · A* : $h = \text{lln} - \text{butIl}$ (euclidienne, admissible)
h admissible (jamais surestimée) → même chemin optimal

Nœuds explorés (lancez les deux !)

Dijkstra : — A* : —

Cliquez ► pour lancer la recherche

Dijkstra

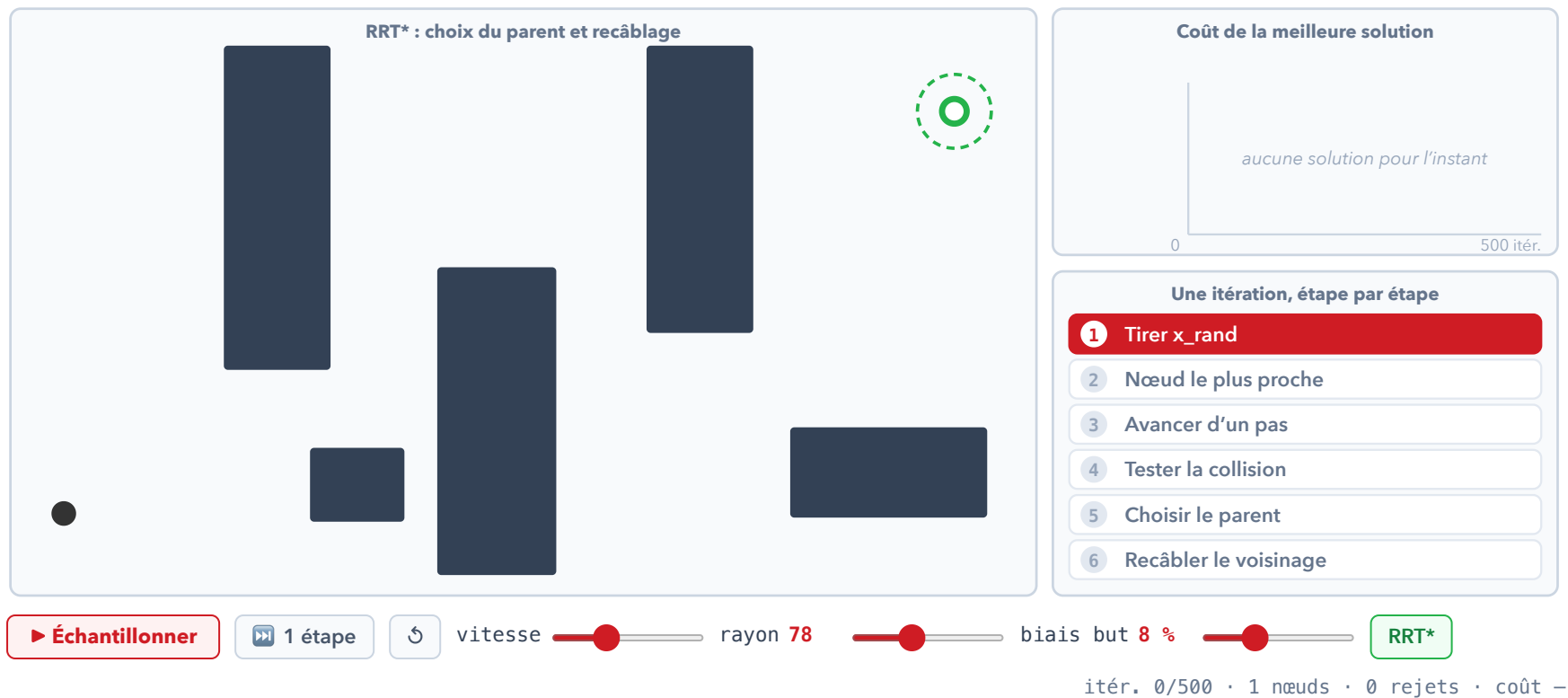
A*

r_robot 0.0

► Chercher

↺

RRT* – Visualisation Interactive



Un point uniforme dans l'espace libre – ou le but lui-même, avec le biais choisi.

Inflation d'Obstacles et Marge de Sécurité

Planifier pour un robot **de rayon r** dans la carte réelle $\equiv$ planifier pour un robot **ponctuel** dans une carte où les obstacles sont **gonflés de r** .

Avec un champ de distance, l'inflation devient un seuil :

$$\text{interdit}(\mathbf{x}) \iff d(\mathbf{x}) \leq r_{\text{robot}} + m$$

m : marge pour la discrétisation et l'incertitude de la carte.

Pas de dilatation morphologique à recalculer : un simple **seuil sur l'ESDF**, ajustable à la volée (autre robot, autre marge).

Au-delà du seuil fixe : les costmaps peuvent ajouter une zone de coût décroissant ($\propto e^{-\alpha(d-r)}$). Le chemin préfère rester loin des murs sans que ce soit interdit.

Robots non circulaires

Pour une empreinte rectangulaire (voiture), l'inflation par un seul rayon est conservatrice. Solutions : plusieurs cercles couvrants, ou vérification d'empreinte exacte sur les cellules critiques (nav2 : footprint).

Marge trop grande $\rightarrow$ couloirs "infranchissables" ; trop petite $\rightarrow$ collisions. C'est un **paramètre de sécurité**, à valider expérimentalement.

Optimisation de Trajectoire sur l'ESDF

Représentation : trajectoire discrétisée en N waypoints

$\mathbf{q}_1, \dots, \mathbf{q}_N$ ($\mathbf{q}_1$ = départ, $\mathbf{q}_N$ = arrivée, fixes). On part d'un tracé grossier et on le **déforme**.

Coût total à minimiser :

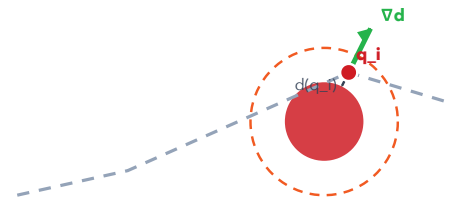
$$J(\mathbf{q}) = \underbrace{\lambda_s \sum_i \|\mathbf{q}_{i+1} - \mathbf{q}_i\|^2}_{\text{lissage}} + \underbrace{\lambda_c \sum_i c(d(\mathbf{q}_i))}_{\text{collision}}$$

$d(\mathbf{q}_i)$: ESDF au waypoint $\mathbf{q}_i$ (distance signée au plus proche obstacle) · $\lambda_s, \lambda_c > 0$: poids lissage/évitement. Le lissage est minimal en ligne droite → il pénalise les zigzags.

Pénalité hinge (nulle loin des obstacles) :

$$c(d) = \begin{cases} (\epsilon - d)^2 & d < \epsilon \\ 0 & d \geq \epsilon \end{cases}$$

ϵ : marge de sécurité – le coût croît **quadratiquement** à l'approche d'un obstacle (jusqu'à $d < 0$ = collision).



pointillé orange = marge ϵ · $\mathbf{q}_i$ poussé le long de $+\nabla d$

Le gradient :

$$\nabla_{\mathbf{q}_i} c = c'(d) \nabla d(\mathbf{q}_i) = -2(\epsilon - d) \nabla d(\mathbf{q}_i)$$

$\nabla d(\mathbf{q}_i)$: gradient de l'ESDF en ce point, pointe **loin** de l'obstacle le plus proche.

La mécanique de mise à jour

Cette recette (lissage + pénalité sur l'ESDF) est au cœur de méthodes comme CHOMP ou TrajOpt. La slide suivante détaille comment ce gradient met à jour chaque waypoint, itération après itération.

Descente de Gradient : Mise à Jour des Waypoints

Règle de mise à jour (un pas de descente de gradient) :

$$\mathbf{q}_i \leftarrow \mathbf{q}_i - \eta \nabla_{\mathbf{q}_i} J(\mathbf{q})$$

η : taille du pas (*learning rate*) – trop grand $\rightarrow$ oscillations, trop petit $\rightarrow$ convergence lente.

Le gradient total $\nabla_{\mathbf{q}_i} J$ est la somme de **deux forces** appliquées à chaque waypoint :

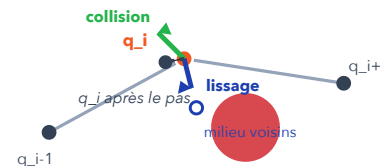
1. Gradient de lissage ($\mathbf{q}_i$ apparaît dans 2 termes de la somme, avec $\mathbf{q}_{i-1}$ et $\mathbf{q}_{i+1}$) :

$$\nabla_{\mathbf{q}_i}(\text{lissage}) = 2(2\mathbf{q}_i - \mathbf{q}_{i-1} - \mathbf{q}_{i+1})$$

Ce terme s'annule quand $\mathbf{q}_i$ est exactement **au milieu** de ses deux voisins : il tire chaque waypoint vers la moyenne de ses voisins, comme une corde qui se tend.

2. Gradient de collision (vu à la slide précédente), actif seulement si $d < \epsilon$:

$$\nabla_{\mathbf{q}_i}(\text{collision}) = -2\lambda_c(\epsilon - d) \nabla d(\mathbf{q}_i)$$



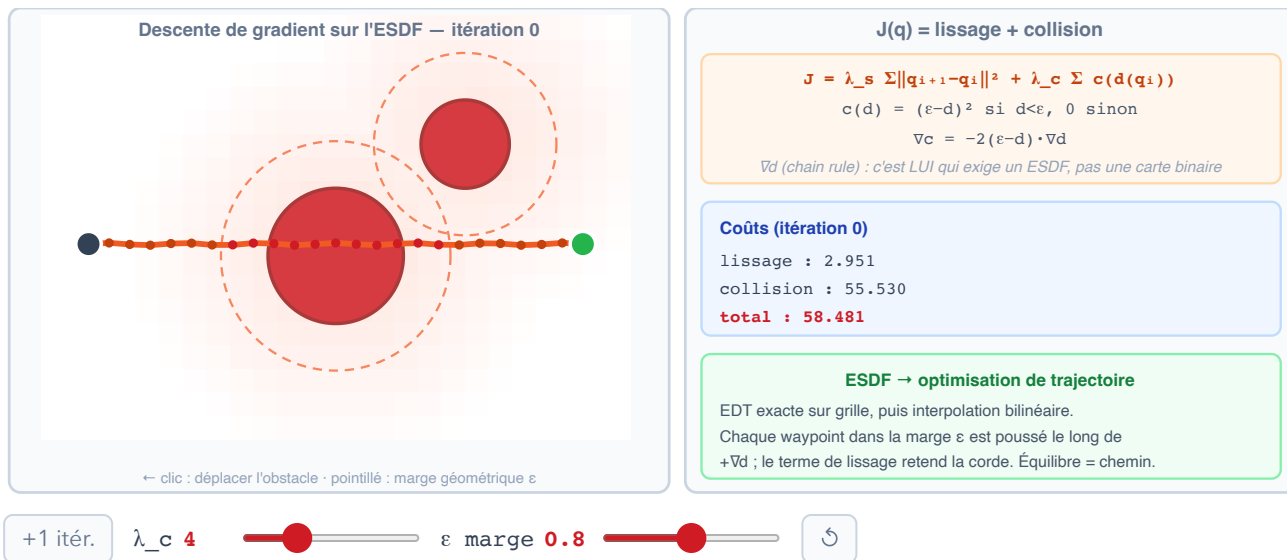
Le déplacement net est la **somme vectorielle** des deux forces : la trajectoire se lisse et s'éloigne des obstacles **simultanément**, un petit pas à la fois.

Minima locaux

La descente de gradient peut rester coincée (obstacle en U). Pipeline standard : **A*/RRT pour l'initialisation** grossière $\rightarrow$ optimisation pour le raffinement. Les deux mondes sont complémentaires.

Optimisation de Trajectoire – Visualisation Interactive

Vraie descente de gradient : les waypoints dans la marge ϵ sont poussés le long de $+\nabla d$ (ESDF interpolé), le lissage retend la trajectoire.



Exploration par Frontières

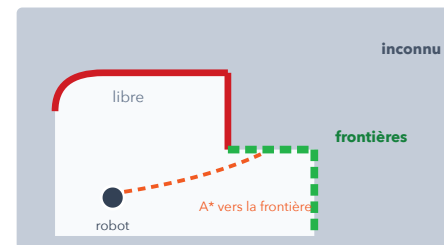
Le problème inverse : jusqu'ici la carte existait. Comment la **construire efficacement** ? Où envoyer le robot ?

Frontière (Yamauchi, 1997) : cellule **libre** adjacente à au moins une cellule **inconnue**. C'est là que de nouvelles mesures apporteront de l'information.

Stratégie greedy (glouton) :

1. Détecter les frontières (balayage de la grille)
2. Aller à la frontière la plus proche (A*)
3. Scanner → la carte s'étend → recommencer
4. Plus de frontières → exploration **terminée**

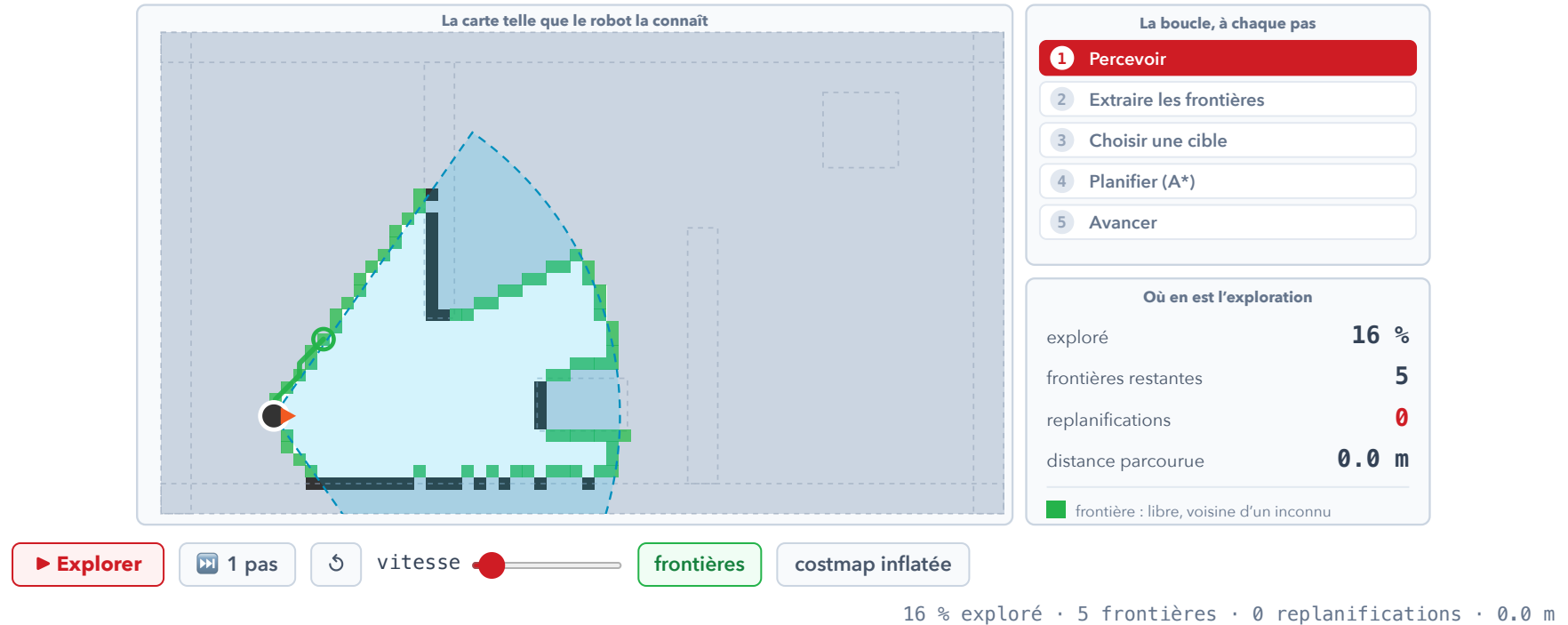
Variantes : gain d'information attendu, coût-utilité, exploration multi-robots (partage des frontières).



L'exploration exige de distinguer **libre / occupé / inconnu**. Une carte d'occupation distingue ces états. Dans un TSDF, $W = 0$ signale une absence d'observation ; ce masque doit être transmis aux conversions et au planificateur.

Exploration par Frontières – Animation

Le robot découvre la carte, choisit une **frontière**, puis **replanifie** lorsque de nouveaux obstacles apparaissent.



Percevoir, extraire les frontières, choisir, planifier, avancer – puis recommencer, à chaque pas.

Planification optimiste : l'inconnu est supposé traversable.

Gris : inconnu · blanc : libre · noir : obstacle · cases vertes : frontières · ligne verte : chemin · cône bleu : champ de vision.

👉 Checkpoint – Composez la Stack Complète

Représentation(s) + structure de données + planificateur, pour chaque robot :

Aspirateur robot

→ **Grille d'occupation 2D** dense (surface bornée) + **A*** sur costmap avec inflation.

Le monde est un plan : la 3D serait du gaspillage.

Drone d'inspection de pont

→ **TSDf** (voxel hashing, domaine extensible) → **maillage** pour le rapport d'inspection, + **ESDF** pour naviguer autour de la structure.

Rover sur terrain accidenté

→ **Carte d'élévation 2.5D** : gradients de pente pour la traversabilité, très léger.

Un tunnel sur l'itinéraire ? → passer en **MLS**.

Le raisonnement complet : **(robot, environnement, tâche)** → Q1 quantité → Q2 abstraction → Q3 structure → planificateur.

Fin du Cours – Récapitulatif

Représenter et stocker

- **Occupation** : libre, occupé, inconnu ; fusion en log-odds.
- **Élévation / MLS, primitives, sémantique** : adapter le contenu à la tâche.
- **Voxels, hachage, octrees** : organiser les données.

Fusionner et convertir

- **ESDF** : transformée euclidienne par passes.
- **TSDF** : fusion pondérée de mesures de profondeur.
- **Marching Cubes** : extraire un maillage triangulé.

Utiliser la carte

- **Environnement dynamique** : éviter les fantômes.
- **A*, RRT*** : chercher un chemin.
- **ESDF** : marge de sécurité et optimisation de trajectoire.
- **Frontières** : explorer l'espace inconnu.

Le fil conducteur

Quantité → représentation → stockage → action.

Le choix dépend du robot, de l'environnement et de la tâche.

Prochains cours : représentations apprises pour la perception et la cartographie.